Universitatea Transilvania din Brașov

Universitatea Transilvania din Brașov
FACULTATEA DE INGINERIE ELECTRICĂ ȘI ȘTIINȚA CALCULATOARELOR

Departamentul : Inginerie Electronică, Telecomunicații și Tehnologii Informaționale

Programul de studii: SECI-SI

BRUJBEANU Gabriel - Lucian

CANDU Ion

PROIECT DE VERIFICARE

Brașov, 2026

Departamentul: Inginerie Electronică, Telecomunicații și Tehnologii Informaționale
Programul de studii: SECI-SI

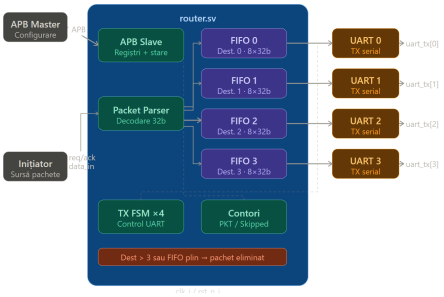
BRUJBEANU Gabriel - Lucian
CANDU Ion

Proiectarea și implementarea unui router de pachete multi-port cu interfață APB în SystemVerilog

Brașov, 2022

FIȘA PROIECTULUI DE VERIFICARE

<table>
<tr><td>Universitatea Transilvania din Braşov</td><td rowspan="2">Anul universitar:
2025 - 2026</td></tr>
<tr><td>Facultatea de Inginerie Electrică şi Ştiința Calculatoarelor</td></tr>
<tr><td>Membrii:
BRUJBEANU Gabriel - Model de referinta, Inginer de verificare, Arhitect
Candu Ion - Inginer de proiectare, Inginer de verificare, Model de referinta

Coordonatori:
Alexandru Dinu</td><td> 
</td></tr>
<tr><td colspan="2">Titlul proiectului: Proiectarea și implementarea unui router de pachete multi-port cu interfață APB în SystemVerilog</td></tr>
<tr><td colspan="2">Sumarul funcționalităților DUT-ului:
◘ Recepționează pachete de 32 de biți pe interfața router (req/ack handshake) și le parsează conform structurii definite.
◘ Rutează pachetele în unul din cele 4 FIFO-uri (câte unul per port de destinație 0–3); pachetele cu destinație > 3 sunt eliminate.
◘ Transmite conținutul FIFO-urilor prin 4 instanțe UART independente, fiecare cu FSM propriu de control TX.
◘ Interfața APB (8-bit addr, 8-bit data) permite configurarea baudrate-ului UART, citirea contorilor de pachete/skipped și a statusului FIFO-urilor.</td></tr>
</table>

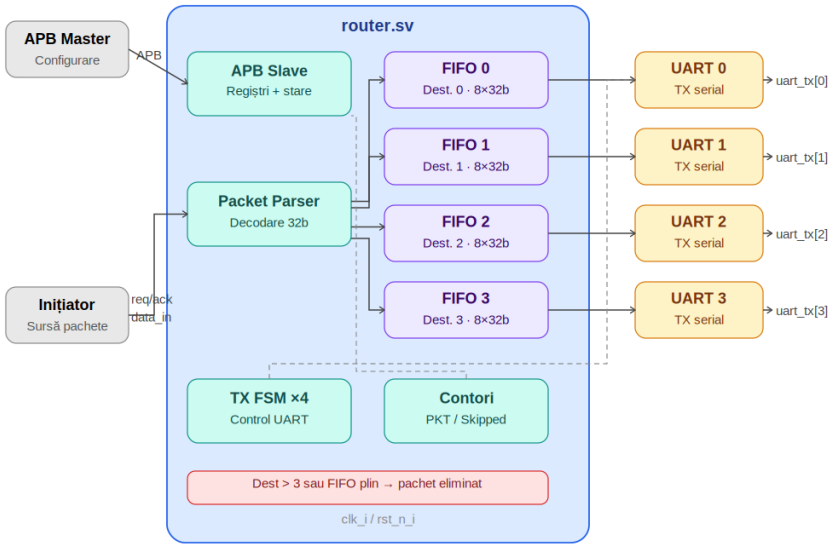


Figura 1. Diagrama bloc a router-ului APB multi-port

Sumarul scenariilor de verificare:

- Verificarea rutării corecte a pachetelor la porturile 0–3.
- Verificarea eliminării pachetelor cu destinație invalidă (> 3).
- Verificarea comportamentului la FIFO plin (packet skipped).
- Verificarea protocoalelor APB (read/write, erori de adresă, semnale stabile).
- Verificarea transmisiei UART și configurarea baudrate prin APB.

Istoricul versiunilor:

Versiune	Data finalizării	Editorul	Motiv
0.0	20.03.2021	Alexandru Dinu	Prima versiune de model
0.1	15.05.2024	28.03.2022	Template-ul a fost actualizat
1.0	24.03.2026	Brujbeanu Gabriel-Lucian	Versiunea inițială

Cuprins

Cuprins
1 6
1.1 Eroare! Marcaj în document nedefinit.
1.2 Eroare! Marcaj în document nedefinit.
1.3 Eroare! Marcaj în document nedefinit.
2 7
2.1 7
2.2 8
2.3 Eroare! Marcaj în document nedefinit.
2.4 9
2.5 Eroare! Marcaj în document nedefinit.
2.5.1 Eroare! Marcaj în document nedefinit.
2.5.2 Eroare! Marcaj în document nedefinit.
3 Eroare! Marcaj în document nedefinit.
3.1 16
3.2 Eroare! Marcaj în document nedefinit.
3.3 Eroare! Marcaj în document nedefinit.
4 Eroare! Marcaj în document nedefinit.
4.1 Eroare! Marcaj în document nedefinit.
4.2 Eroare! Marcaj în document nedefinit.
5 Eroare! Marcaj în document nedefinit.
5.1 Eroare! Marcaj în document nedefinit.
5.2 Eroare! Marcaj în document nedefinit.
5.3 Eroare! Marcaj în document nedefinit.
6 Eroare! Marcaj în document nedefinit.
6.1 Eroare! Marcaj în document nedefinit.
6.2 Eroare! Marcaj în document nedefinit.

LOCAȚIA PROIECTULUI

Git: https://github.com/Candu34/VLSI_Router

EDA Playground: https://edaplayground.com/x/6mcW

IERARHIA FIȘIERELOR

```
router.sv        — modul top-level (APB + parser + FSM TX)
fifo_memory.sv   — memorie FIFO parametrizabilă (SIZE, WIDTH)
uart.sv          — modul UART cu baudrate configurabil
tb/              — directorul testbench-ului
```

MODULUL DE TOP

Modulul de top este router.sv, care instanțiază și interconectează toate componentele: 4 instanțe fifo_memory și 4 instanțe UART. Acesta expune interfața APB pentru configurare și interfața router (req/ack) pentru recepția pachetelor de date.

1 PROIECTUL DIGITAL AL CIRCUITULUI

Diagrama bloc

Descrierea funcționalității generale a DUT-ului

Interfețele DUT-ului

Lista cu funcționalitățile DUT-ului

Schema de principiu

Forme de undă

Tabel cu regiștri

1.1 DIAGRAMA BLOC

Figura 2 prezintă diagrama bloc a router-ului de pachete. Pachetele intră prin interfața Router (req/ack), sunt parsate de Packet Parser și direcționate în câte un FIFO dedicat fiecărui port de destinație (0–3). Fiecare FIFO alimentează un modul UART care transmite datele serial. Interfața APB oferă acces la registrele de configurare și stare.

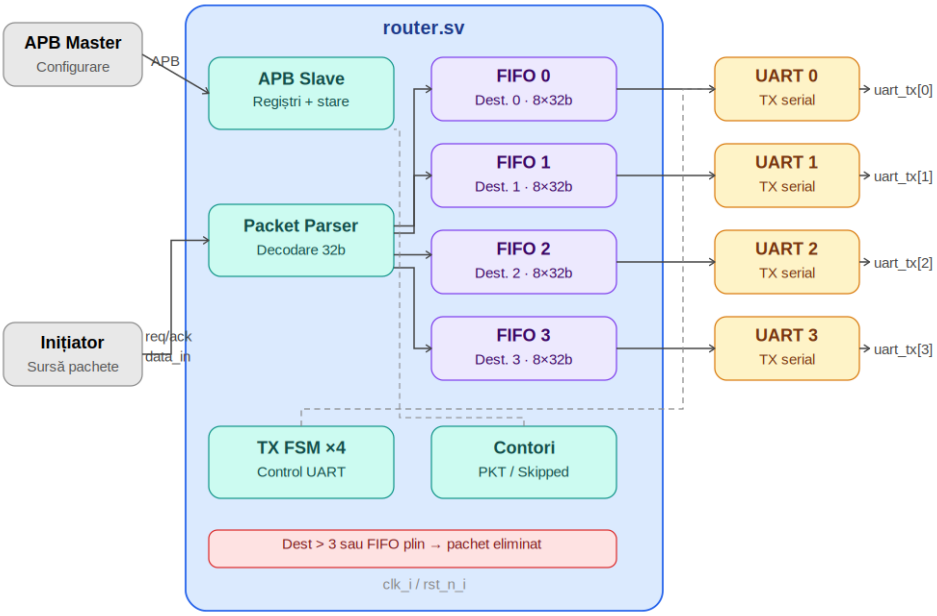


Figura 2. Diagrama bloc – Router APB multi-port

1.2 DESCRIEREA FUNCȚIONALITĂȚII GENERALE A DUT-ULUI

Router-ul recepționează pachete de 32 de biți prin interfața router (data_in/req_i/ack_o). Fiecare pachet este decodat conform structurii packet_t definite în SystemVerilog:

- biții [31:28] — adresa de destinație (0–3 valid; > 3 → pachet eliminat);
- biții [27:24] — adresa sursă;
- biții [23:22] — prioritatea (0=low, 1=normal, 2=high, 3=urgent);
- biții [21:20] — tipul pachetului (00=data, 01=control, 10=status, 11=error);
- biții [19:16] — numărul de secvență (0–15);

Pachetele valide sunt înscrise în FIFO-ul corespunzător portului de destinație. Un FSM de control TX (câte unul per canal) configurează și controlează transmisia UART. Interfața APB permite configurarea baudrate-ului și citirea registrelor de stare.

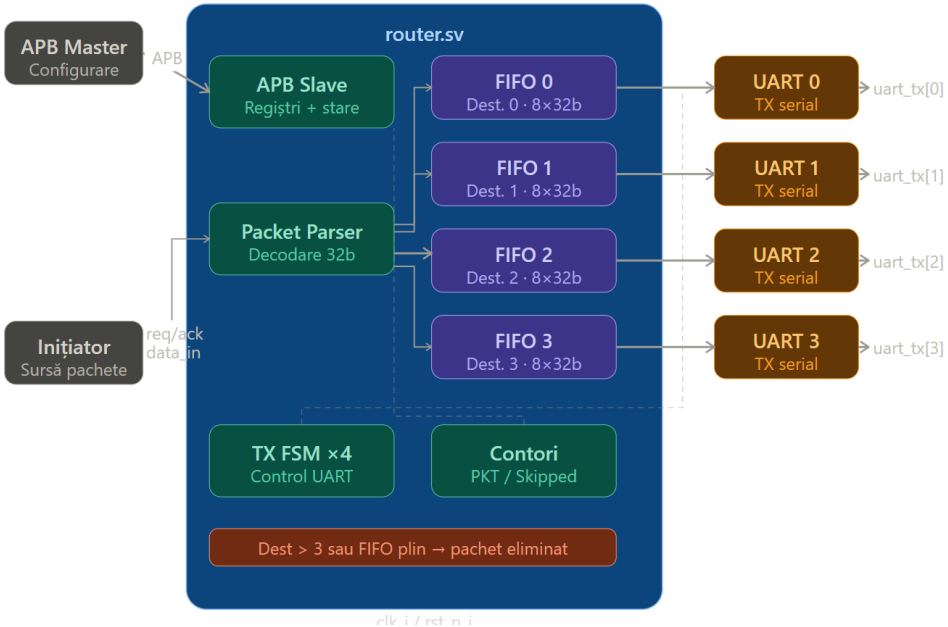


Figura 3. Diagrama generală – Referință funcționalitate DUT

1.3 INTERFEȚELE DUT-ULUI

Tabelul 1. INTERFAȚA DE SISTEM

NUME	PORT	DIMENSIUNE (biți)	FUNCȚIONALITATE
clk_i	I	1	Semnal de ceas
rst_n_i	I	1	Reset asincron, activ pe 0 (active-low)

Tabelul 2. INTERFAȚA APB

NUME	PORT	DIMENSIUNE (biți)	FUNCȚIONALITATE
psel_i	I	1	Selectare slave APB
penable_i	I	1	Enable – indică al doilea ciclu APB
pr_w_i	I	1	Direcție (1=write, 0=read)
paddr_i	I	8	Adresă APB
pwdata_i	I	8	Date de scris
prdata_o	O	8	Date citite
p_error_o	O	1	Eroare (adresă invalidă / write read-only)
p_ready_o	O	1	Ready – extinderea transferului APB

Tabelul 3. INTERFAȚA ROUTER

NUME	PORT	DIMENSIUNE (biți)	FUNCȚIONALITATE
data_in	I	32	Pachetul de intrare (structurat packet_t)
req_i	I	1	Request – data_in validă
ack_o	O	1	Acknowledge – router-ul a acceptat pachetul

Tabelul 4. INTERFAȚA UART TX

NUME	PORT	DIMENSIUNE (biți)	FUNCȚIONALITATE
uart_tx_o[3:0]	O	4	Ieșirile seriale TX (câte una per port de destinație)

1.4 LISTA CU FUNCȚIONALITĂȚILE DUT-ULUI

Ft. 1: Recepționează pachete de 32 de biți pe interfața req/ack și le parsează conform structurii packet_t.

Ft. 2: Rutează pachetele în funcție de câmpul dest_addr[31:28] la unul din cele 4 FIFO-uri de ieșire (dest. 0–3).

Ft. 3: Elimină pachetele cu destinație invalidă (dest_addr >= 4) și incrementează contorul PKT_SKIP_CNT.

Ft. 4: Buffering în 4 memorii FIFO independente (8 intrări × 32 biți fiecare), câte una per port de destinație.

Ft. 5: Transmisie serială prin 4 instanțe UART cu baudrate configurabil și FSM TX per canal (TX_IDLE → TX_CFG_BAUD → TX_SEND → TX_WAIT_DONE).

Ft. 6: Configurare și monitorizare prin APB: baudrate UART (0x01), contor pachete (0x03), contor pachete eliminate (0x04), status FIFO (0x05).

Ft. 7: Reset asincron active-low (rst_n_i) pentru toate registrele, FSM-urile și FIFO-urile interne.

1.5 SCHEMA DE PRINCIPIU

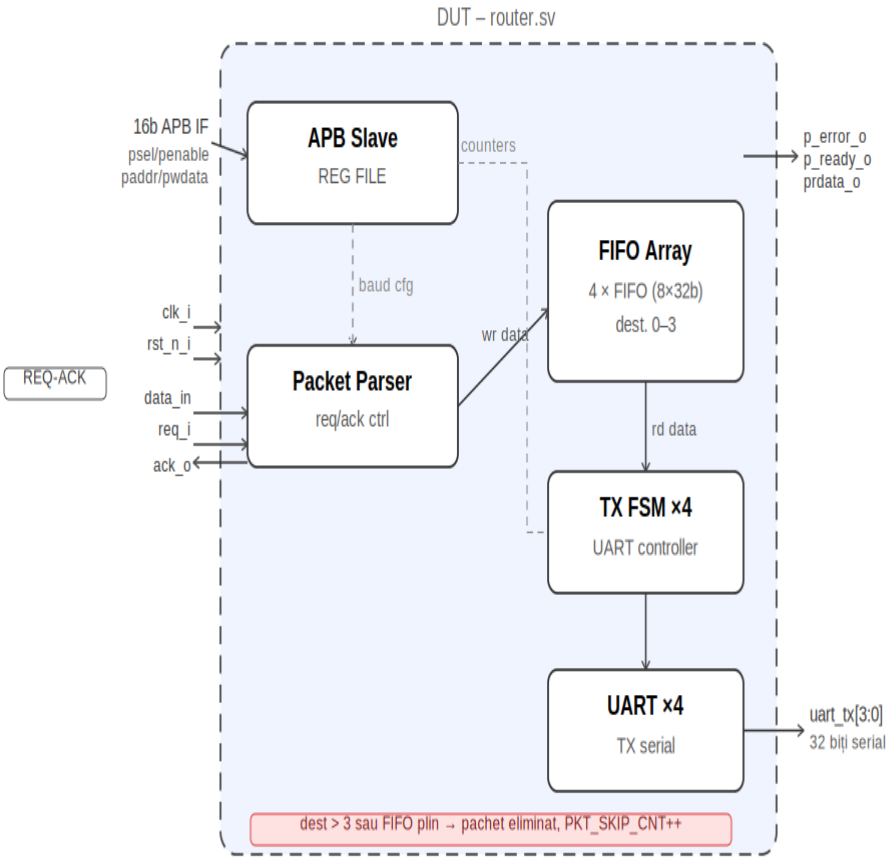


Figura 4. Schema de principiu a router-ului APB multi-port cu interfețele externe

Schema bloc a DUT-ului

Schema bloc detaliată (Figura 5) prezintă interconexiunile interne dintre modulele router-ului: APB Slave pentru configurare, Packet Parser pentru decodarea pachetelor, FIFO Array pentru buffering per port, TX FSM pentru controlul transmisiei și instanțele UART pentru ieșirea serială.

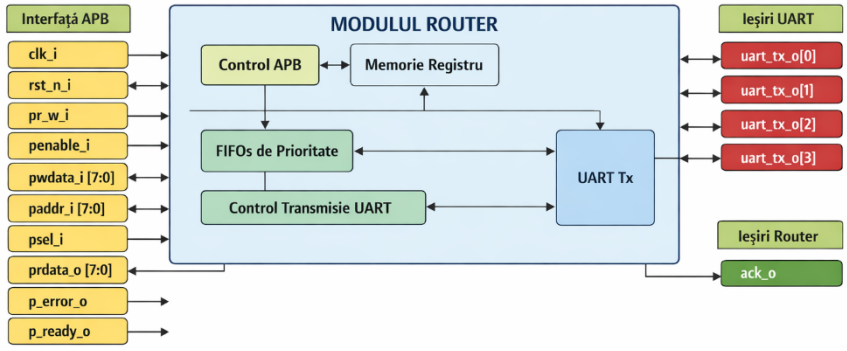


Figura 5. Schema bloc a DUT-ului – router.sv

Schema RTL a DUT-ului

Schema RTL (Register Transfer Level) descrie comportamentul modulelor la nivel de registre și transferuri de date. Principalele blocuri RTL identificate în cod sunt:

- APB Write always block — sincron, actualizează memory[0..2] și semnalizează baud_changed;
- APB Read always block — sincron, scoate pe prdata_o valoarea din memory[paddr_i];
- Packet Parser always block — sincron, parsează data_in prin castare la packet_t, scrie în FIFO;
- ack_o always block — afirmă ack_o pentru un tact la req_i activ;
- PKT_CNT / PKT_SKIP_CNT always blocks — incrementare contori la ack_o respectiv pachet invalid;
- TX FSM generate block — câte un FSM per FIFO, controlează UART-ul;
- FIFO STATUS always block — actualizează memory[5] cu biții full/empty ai FIFO-urilor.

- **Enumerarea și descrierea parametrilor DUT-ului**

  DUT-ul are 4 parametri locali (localparam) definiți în router.sv:

Parametru	Valoare implicită	Descriere
APB_MEMORY_SIZE	6	Numărul de octeți de memorie APB (regiștri interni)
NUM_FIFOS	4	Numărul de FIFO-uri (egal cu porturile de destinație valide)
FIFO_SIZE	8	Capacitatea fiecărui FIFO (intrări × FIFO_WIDTH)
FIFO_WIDTH	32	Lățimea unei intrări FIFO în biți (egal cu lățimea pachetului)

Modulul UART intern are și el parametri impliciti controlați prin registre: lungimea datelor (cfg[2:1]: 00=8b, 01=16b, 10=32b, 11=64b), direcția de transmisie (cfg[0]: 0=LSB first, 1=MSB first) și baudrate-ul (baud[2:0]).

- **Probleme întâmpinate la proiectarea DUT-ului și cum au fost rezolvate acestea**

1. Gestionarea simultană a multiplelor FIFO-uri — soluție: generare parametrizată (generate for) cu instanțiere a câte unui FIFO și UART per port, controlate de FSM-uri independente.

2. Sincronizarea baud_changed — la modificarea baudrate-ului prin APB, toate cele 4 FSM-uri TX trebuie să reconfigureze UART-ul înainte de a trimite date noi. Soluție: flagul baud_changed resetează cfg_done[i] pentru fiecare canal, forțând o trecere prin starea TX_CFG_BAUD.

3. Comportamentul ack_o la FIFO plin — designul curent afirmă ack_o chiar dacă pachetul este eliminat (FIFO plin sau destinație invalidă). Aceasta este o limitare documentată, nu o eroare de proiectare.

4. Lățimea contorilor APB (8 biți) — la mai mult de 255 pachete, contoarele PKT_CNT și PKT_SKIP_CNT se resetează la 0 (wrap-around). Soluție pentru versiuni viitoare: extinderea la 16 biți cu regiștri pereche.

- **Limitări în funcționarea DUT-ului**

  L1. Routerul suportă doar 4 porturi de destinație (0–3); orice altă destinație cauzează eliminarea pachetului.

  L2. FIFO-ul are capacitate de 8 pachete; la depășire, pachetele noi sunt eliminate.

  L3. Interfata APB are latimea de 8 biti pe date si adresa; contoarele (8 biti) se vor wrapa dupa 255 pachete.

  L4. Modulul nu implementează prioritizarea pachetelor pe baza câmpului prio; toate prioritățile sunt tratate egal.

  L5. ack_o este afirmat chiar dacă pachetul este eliminat (FIFO plin sau destinație invalidă).

- **Modul de funcționare al DUT-ului**

  La recepția unui semnal req_i activ, DUT-ul parsează data_in prin castare la tipul struct packet_t. Dacă destinația este validă (< 4) și FIFO-ul corespunzător nu este plin, pachetul este înscris, iar ack_o este ridicat în tactul următor. Dacă destinația este invalidă sau FIFO-ul este plin, pachetul este abandonat și memory[4] (PKT_SKIP_CNT) este incrementat. FSM-ul TX al fiecărui canal urmează pașii: TX_IDLE → TX_CFG_BAUD (la baud_changed) → TX_SEND → TX_WAIT_DONE.

- **Protocolul APB**

  Transferul respectă secvența standard: faza SETUP (psel=1, penable=0) urmată de faza ACCESS (psel=1, penable=1). La scriere, datele de pe pwdata_i sunt stocate în registrul adresat; la citire, valoarea registrului este pusă pe prdata_o. Semnalul p_error_o este ridicat dacă adresa depășește APB_MEMORY_SIZE sau dacă se încearcă scrierea în regiștrii read-only (adrese >= 3).

- **Protocolul Req-Ack (interfața Router)**

  Inițiatorul afirmă req_i și plasează pachetul valid pe data_in. DUT-ul răspunde cu ack_o ridicat pentru un singur tact de ceas. Inițiatorul trebuie să mențină data_in stabil pe durata req_i.

- **Protocolul UART**

  Modulul UART intern implementează transmisia serială asincronă cu format configurabil (8/16/32/64 biți date, LSB sau MSB first, baudrate programabil prin registrul baud[2:0]). Controllerul TX interacționează cu UART prin propriul protocol

intern req/ack: adresa 1 configurează baudrate-ul, adresa 2 declanșează transmisia.

1.6 FORME DE UNDĂ

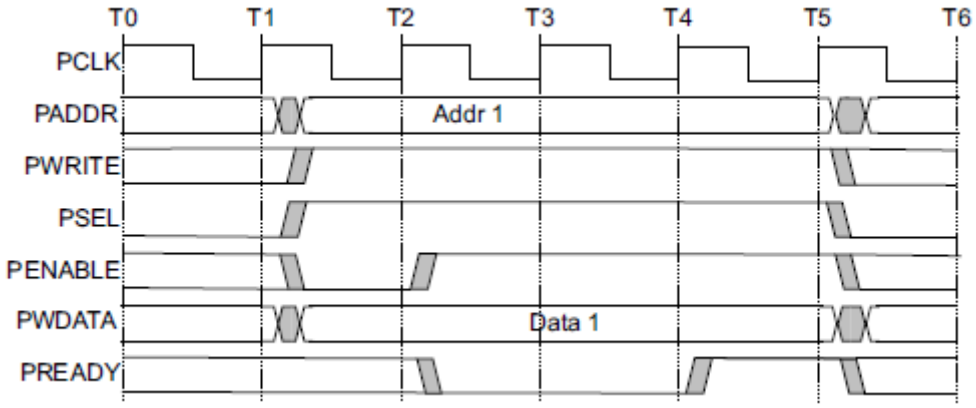


Figura 7. Formele de undă pentru APB write

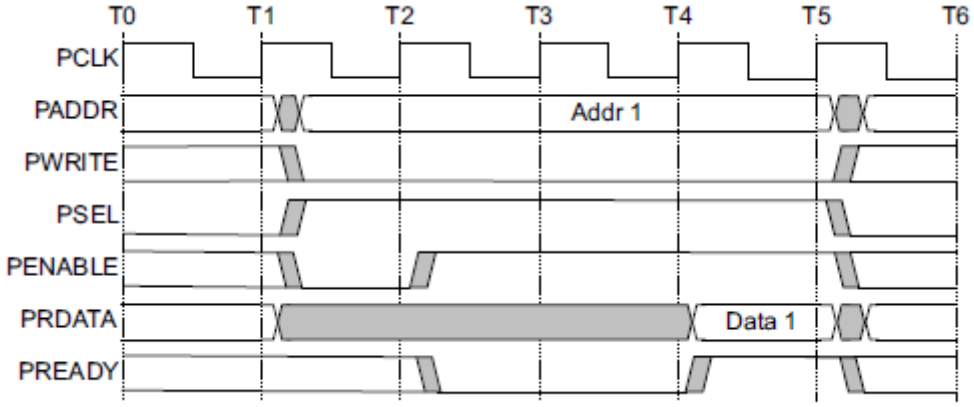


Figura 8. Formele de undă pentru APB read

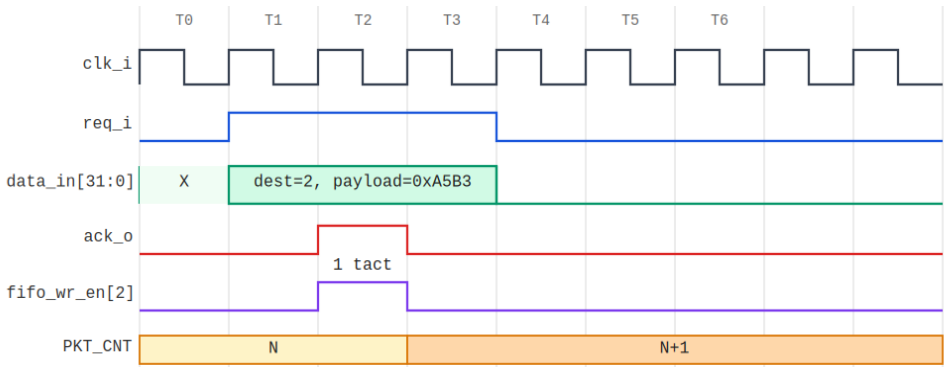


Figura 9. Formele de undă pentru request-acknowledge pentru și scrierea FIFO

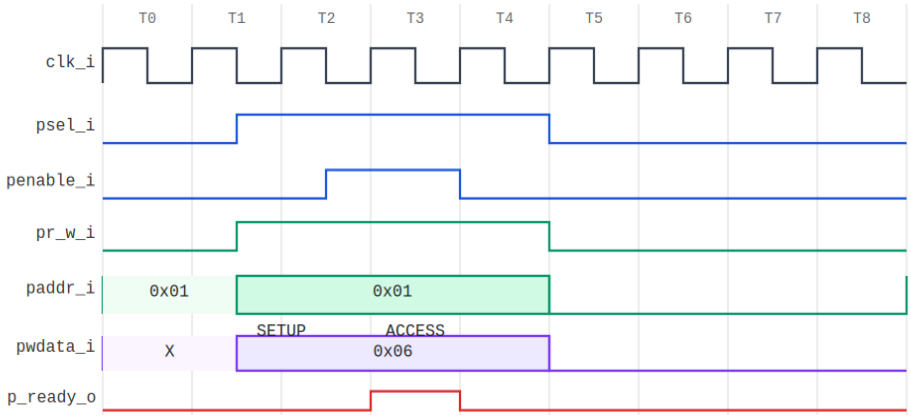


Figura 10. Formele de undă APB write – detaliu faze SETUP și ACCESS

1.7 TABEL CU REGIȘTRI

Tabelul 4. Tabel cu regiștrii care pot fi accesați de către DUT

Adresă	Nume Registru	Acces	Structura / Descriere
0x00	CTRL	R/W	bit[7]: Enable Router; bit[6]: Reset Router; biții [5:0]: rezervați
0x01	UART_CLK_DIV	R/W	biții [2:0]: divizor baudrate UART (aplicat tuturor celor 4 canale)
0x02	Rezervat	—	Nefolosit în implementarea curentă
0x03	PKT_CNT	R	Contor pachete acceptate (incrementat la fiecare ack_o)

Adresă	Nume Registru	Acces	Structura / Descriere
0x04	PKT_SKIP_CNT	R	Contor pachete eliminate (destinație invalidă sau FIFO plin)
0x05	STATUS	R	[7:6] FULL/EMPTY FIFO3; [5:4] FIFO2; [3:2] FIFO1; [1:0] FIFO0

2 VERIFICAREA PROIECTULUI DIGITAL AL CIRCUITULUI

Arhitectura mediului de verificare

Lista cu elementele de verificat

Elementele de acoperire funcțională

Testele

Rezultatele obținute

2.1 ARHITECTURA MEDIULUI DE VERIFICARE

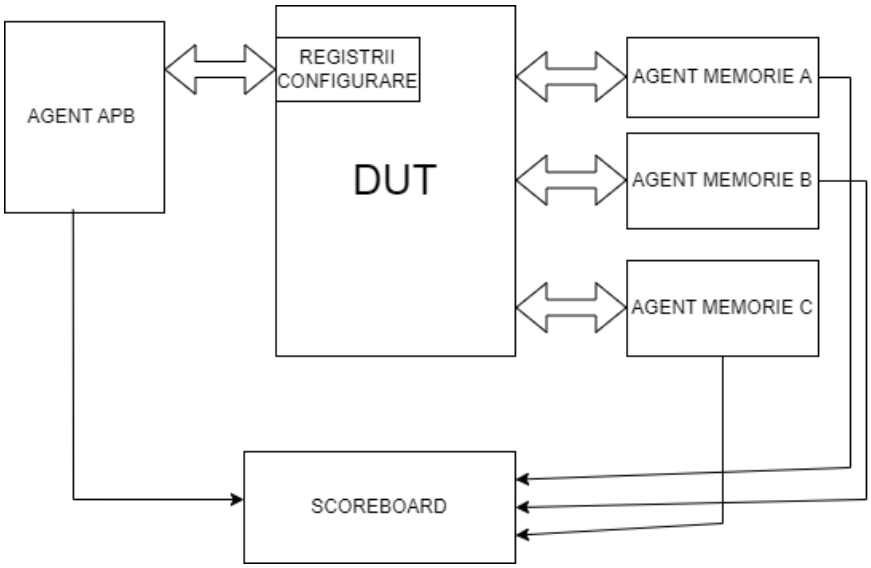


Figura 11. Imaginea de ansamblu a mediului de verificare

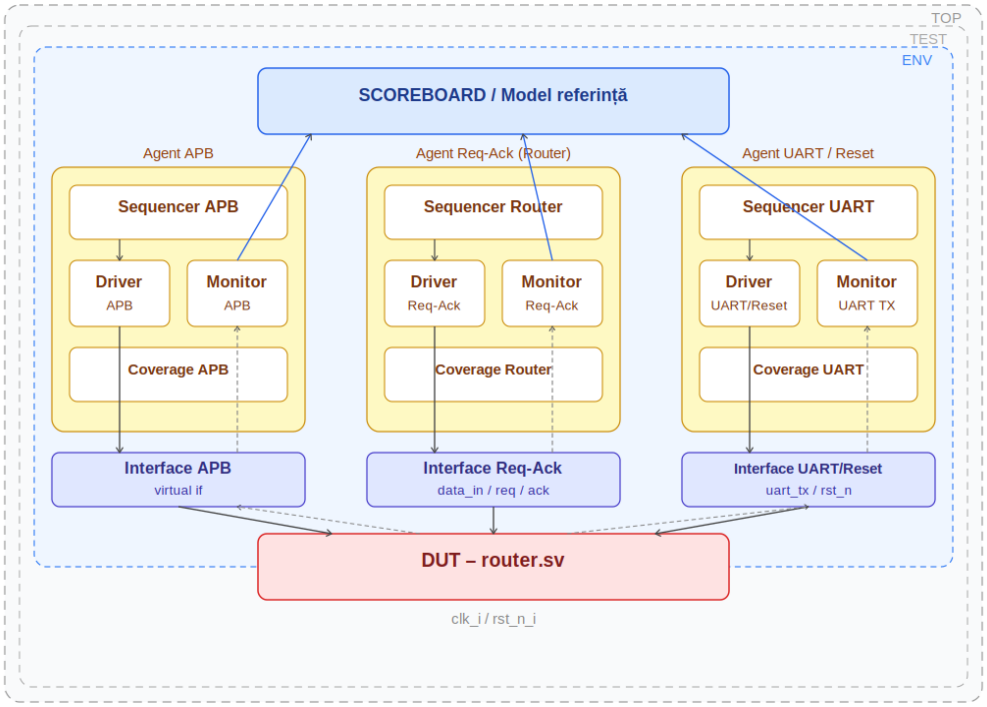


Figura 12: Mediu de verificare: Testbench si test environment

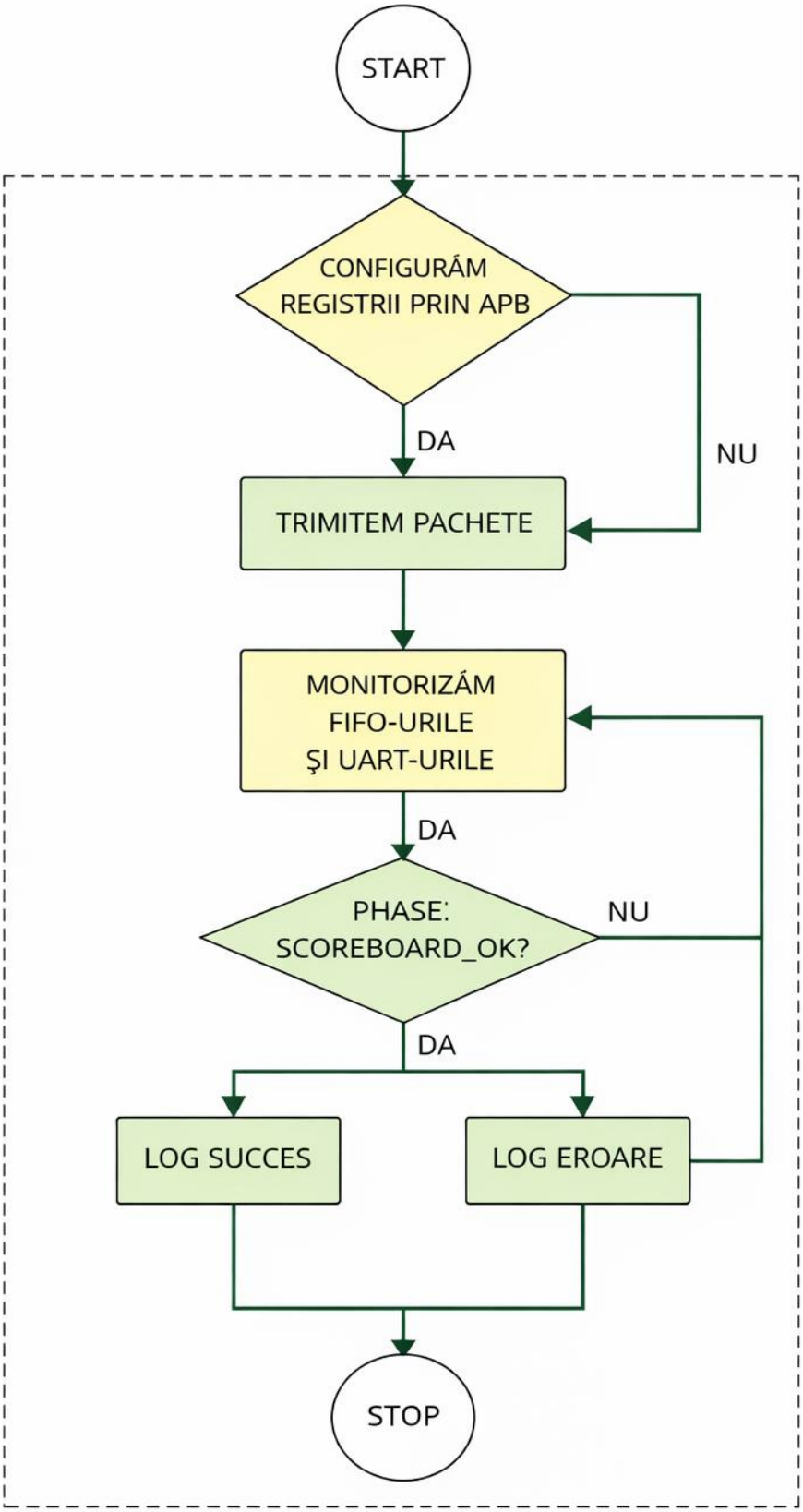

START
CONFIGURĂM
REGISTRII PRIN APB
DA
NU
TRIMITEM PACHETE
MONITORIZĂM
FIFO-URILE
ŞI UART-URILE
DA
PHASE:
SCOREBOARD_OK?
NU
DA
LOG SUCCES
LOG EROARE
STOP

Figura 13. Schemă logică.

- **Driver pentru interfața APB**

Driverul APB este o componentă activă a mediului de verificare care trimite tranzacții (read/write) către DUT respectând specificațiile protocolului APB. Penable_i se asertează la un tact după psel_i. La scriere, pwdata_i primește valoarea din item; la citire, valoarea de pe prdata_o este capturată. Driverul gestionează și cazuri de eroare: adrese invalide, scriere în regiștri read-only.

- **Driver pentru interfața Req-Ack (Router)**

Driverul Router generează pachete de 32 de biți pe interfața req/ack. Acesta menține req_i activ și data_in valid până la recepția ack_o. Poate genera pachete atât cu destinații valide (0–3) cât și invalide (4–15), pachete back-to-back și cu întârzieri configurabile.

- **Driver pentru interfața UART / Reset**

Driverul UART monitorizează liniile uart_tx_o[3:0] și decodifică transmisiile seriale. Driverul de reset aplică rst_n_i conform scenariilor de test, inclusiv reset în timpul procesării unui pachet.

- **Monitor APB**

Monitorul APB este o componentă pasivă care capturează semnalele de pe interfața APB prin interfața virtuală și le traduce în tranzacții transmise scoreboard-ului prin port de analiză TLM (Transaction Level Modeling).

- **Monitor pentru interfața Req-Ack**

Monitorul Req-Ack capturează perechile req/ack și extrage pachetele procesate de DUT. Verifică că ack_o este afirmat exact un tact după req_i și că data_in rămâne stabil pe durata req_i.

- **Monitor UART TX**

Monitorul UART decodifică transmisiile seriale de pe uart_tx_o[3:0] și verifică că datele transmise corespund pachetelor rutate în FIFO-ul corespunzător.

- **Scoreboard / Model de referință**

Scoreboard-ul menține o replică software a stării DUT-ului: 4 FIFO-uri virtuale, registrele APB și contoarele de pachete. La fiecare pachet recepționat, scoreboard-ul calculează destinația așteptată și compară comportamentul DUT cu cel expected. Verifică rutarea corectă, incrementarea corectă a contoarelor PKT_CNT și PKT_SKIP_CNT, și că statusul FIFO-urilor (memory[5]) reflectă starea reală.

- **Agent APB**

Agentul APB încapsulează Sequencer-ul, Driver-ul și Monitorul APB într-o singură entitate, conectând componentele prin interfața virtuală.

- **Agent Req-Ack (Router)**

Agentul Router încapsulează Sequencer-ul, Driver-ul și Monitorul Req-Ack, gestionând generarea și capturarea pachetelor pe interfața router.

- **Agent UART / Reset**

Agentul UART/Reset include monitorizarea liniilor TX și controlul semnalului de reset asincron.

- **Colector de coverage APB**

Colectorul de acoperire APB înregistrează accesarea tuturor adreselor valide și invalide, tipurile de tranzacții (read/write) și valorile scrise în registrul UART_CLK_DIV.

- **Colector de coverage Router**

Colectorul Router înregistrează destinațiile pachetelor (0–3 și invalide), tipurile de pachete (data/control/status/error), nivelurile de prioritate și scenariile de FIFO plin. Verifică acoperirea tuturor cauzelor de skip: destinație invalidă și FIFO plin.

2.2 LISTA CU ELEMENTELE DE VERIFICAT

Checker	Descriere
route_check	Verifică dacă pachetul este înscris în FIFO-ul corespunzător dest_addr
skip_invalid_check	Verifică că PKT_SKIP_CNT crește la destinație invalidă (dest >= 4)
skip_full_check	Verifică că PKT_SKIP_CNT crește la FIFO plin
pkt_cnt_check	Verifică că PKT_CNT crește la fiecare ack_o afirmat
apb_rd_check	Verifică că datele citite prin APB corespund valorilor din regiștri
apb_wr_check	Verifică că scrierea APB modifică corect registrul adresat
uart_baud_check	Verifică că UART-ul este reconfigurat la modificarea UART_CLK_DIV
fifo_status_check	Verifică că biții FULL/EMPTY din memory[5] reflectă starea reală a FIFO-urilor
fifo_order_check	Verifică că pachetele sunt transmise UART în ordinea FIFO (FIFO integrity)
uart_tx_data_check	Verifică că datele transmise serial corespund pachetelor din FIFO
reset_check	Verifică că toate semnalele de ieșire revin la 0 la reset asincron

Checker APB IF	Descriere

idle_state	Daca paddr s-a schimbat inseamna ca transferul s-a terminat (la aceiasi adresa) sau nu este un transfer valid
setup_state	Verifica daca penable s-a ridicat in urmatorul tact dupa ce psel a fost ridicat
access_wait_state	Verifica daca in timpul unui acces cu timpi de asteptare psel si penable raman HIGH si doar pready face toggle.
pr_generic_stable(signal)	Verifica daca semnalele sunt stabile in timpul unui transfer
pr_generic_not_unknown (signal)	Checker pentru a asigura ca semnalele nu sunt in X (Warning message)
pwdata_in_read_transfer	Verifica daca pwdata se schimba in timpul unui transfer de citire
prdata_in_wr_transfer	Verifica daca prdata se schimba in timpul unui transfer de scriere
penable_in_transfer	Penable trebuie sa ramana HIGH pe tot timpul transferului.
psel_stable_in_transfer	Psel trebuie sa fie stabil (HIGH) in timpul transferului atunci cand penable este stabil (HIGH) in tot timpul unui transfer.

Checker REQ-ACK IF (MEM IF)	**Descriere**
pr_generic_not_unknown (signal)	Checker pentru a asigura ca semnalele nu sunt in X (Warning message)
ack_pulse	Verifica daca semnalul acknowledge este un puls.
req_without_ack	Verifica situatia in care ai un request fara acknowledge venit, request-ul pica fara acknowledge
ack_without_req	Verifica daca acknowledge se activeaza fara un request activ.
ack_after_one_req_cc	Verifica ca acknowledge sa nu vina in acelasi tact cand request-ul s-a ridicat.

Checker REQ-ACK / FIFO	Descriere
ack_one_cycle	ack_o este afirmat exact un singur tact după req_i

Checker REQ-ACK / FIFO	Descriere
data_stable_during_req	data_in rămâne stabil pe durata req_i
no_wr_when_full	fifo_wr_en nu este ridicat dacă FIFO-ul destinație este plin
fifo_empty_after_reset	Toate FIFO-urile sunt goale imediat după rst_n_i = 0

2.3 ELEMENTELE DE ACOPERIRE FUNCȚIONALĂ

Cover Point	Descriere
dest_cp	Toate cele 4 destinații valide (0–3) + destinații invalide (4..15) testate
prio_cp	Toate cele 4 niveluri de prioritate (low/normal/high/urgent) testate
pkt_type_cp	Toate cele 4 tipuri de pachete (data/control/status/error) testate
fifo_full_cp	Fiecare din cele 4 FIFO-uri a atins starea FULL
fifo_empty_cp	Fiecare FIFO a atins starea EMPTY (golire completă)
apb_addr_cp	Toate adresele valide (0x00–0x05) accesate + adrese invalide
apb_rw_cp	Ambele tipuri de transfer APB (read și write) acoperite per registru
baud_cp	Toate valorile baudrate (0–7) configurate prin UART_CLK_DIV
skip_cause_cp	Ambele cauze de skip: destinație invalidă + FIFO plin – acoperite
back_to_back_cp	Pachete consecutive (fără pauză între req_i)
reset_during_op_cp	Reset aplicat în timpul procesării unui pachet (rst_n_i mid-operation)
uart_cfg_change_cp	Baudrate schimbat în timp ce un canal UART transmitea
checkers_cp	Toate checker-ele au trecut (PASS) cel puțin o dată

2.4 TESTELE

Tabelul 5. Lista testelor implementate pentru verificarea DUT-ului

Numele testului	Descrierea testului
apb_wr_baud_test	Configurarea baudrate-ului (UART_CLK_DIV) prin APB write și citirea înapoi
apb_rd_all_regs_test	Citirea tuturor celor 6 regiștri APB și verificarea valorilor implicite după reset
apb_invalid_addr_test	Accesarea adreselor invalide (paddr >= 6); verificarea p_error_o
apb_ro_write_test	Scrierea în regiștri read-only (0x03, 0x04, 0x05); verificarea p_error_o
route_port0_test	Trimiterea unui pachet cu dest=0; verificarea înscrierii în FIFO 0
route_port1_test	Trimiterea unui pachet cu dest=1; verificarea înscrierii în FIFO 1
route_port2_test	Trimiterea unui pachet cu dest=2; verificarea înscrierii în FIFO 2
route_port3_test	Trimiterea unui pachet cu dest=3; verificarea înscrierii în FIFO 3
route_all_ports_test	Câte un pachet pentru fiecare port valid (0–3) în secvență
invalid_dest_test	Pachete cu dest_addr in {4..15}; verificarea PKT_SKIP_CNT si ca niciun FIFO nu e scris
fifo_overflow_test	Umplerea unui FIFO la 8 pachete + 1 suplimentar; verificarea skip și FULL bit in STATUS
fifo_status_test	Verificarea registrului STATUS (memory[5]) la FULL și EMPTY pentru fiecare FIFO
pkt_cnt_test	Verificarea că PKT_CNT se incrementează corect la fiecare ack_o

Numele testului	Descrierea testului
reset_async_test	Aplicarea rst_n_i la jumătatea procesării unui pachet; verificarea stării post-reset
uart_tx_baud_test	Verificarea că TX FSM configurează UART-ul înainte de transmisie la baud_changed
uart_tx_data_test	Verificarea că datele transmise serial corespund pachetelor din FIFO
back_to_back_test	Pachete consecutive fără pauze (req_i imediat după ack_o); verificarea tuturor
baud_change_during_tx_test	Schimbarea baudrate-ului (APB write) în timp ce un canal transmitea
multi_port_concurrent_test	Pachete simultane pe toate cele 4 porturi; verificarea independenței FIFO-urilor
random_directed_test	N iterații cu pachete randomizate (dest, prio, pkt_type, payload); verificare scoreboard
random_stress_test	Stress test cu burst mare de pachete randomizate inclusiv destinații invalide

2.5 LIMITARI

L1. Testbench-ul verifică comportamentul funcțional; nu include verificare de timing sau constrângeri de frecvență.

L2. Decodificarea UART TX presupune că monitorul cunoaște a priori configurația baudrate și lungimea datelor.

L3. Nu se testează comportamentul la overflow al contoarelor APB (PKT_CNT, PKT_SKIP_CNT) — limitare recunoscută a designului.

Interfețele

- **Interfețele utilizate și rolul acestora**

DUT-ul expune trei interfețe principale. Rolul fiecăreia:

- Interfața APB — configurare și monitorizare; permite accesul la harta de regiștri (baudrate, status, contori);
- Interfața Router (Req-Ack) — recepția pachetelor de 32 de biți de la inițiator;

- Interfața UART TX — 4 linii seriale independente de ieșire, câte una per port de destinație.

Aserțiile

Aserțiunea	Descriere
ack_one_cycle	ack_o este afirmat exact un singur tact după req_i
apb_setup_before_access	penable_i trebuie ridicat la un tact după psel_i
apb_psel_stable	psel_i rămâne HIGH pe toată durata transferului
apb_prdata_stable_wr	prdata_o nu se schimbă în timpul unui transfer de scriere
apb_error_invalid_addr	p_error_o este ridicat dacă paddr_i >= APB_MEMORY_SIZE
apb_error_ro_write	p_error_o este ridicat la scriere în regiștri read-only (paddr >= 3)
fifo_no_wr_when_full	fifo_wr_en nu este ridicat dacă FIFO-ul corespunzător este plin
fifo_no_rd_when_empty	fifo_rd_en nu este ridicat dacă FIFO-ul corespunzător este gol
reset_clears_ack	La rst_n_i=0, ack_o revine la 0 în tactul următor
reset_clears_error	La rst_n_i=0, p_error_o revine la 0 în tactul următor

Concluzii generale și direcții viitoare

- Concluzii generale

Proiectul implementează un router de pachete multi-port cu 4 porturi de ieșire, bufferizare FIFO și transmisie UART, configurabil prin interfața APB. Arhitectura este modulară și scalabilă: creșterea numărului de porturi necesită doar modificarea parametrului NUM_FIFOS și adăugarea instanțelor corespunzătoare de FIFO și UART. Interfața APB oferă o bună observabilitate a stării interne prin registrele de stare și contoare.

Mediul de verificare acoperă principalele scenarii funcționale: rutare corectă, eliminare pachete invalide, comportament la FIFO plin, protocoale APB și UART, și reset asincron.

- Direcții viitoare
- Implementarea prioritizării pachetelor pe baza câmpului prio[23:22] — un scheduler care servește mai întâi FIFO-urile cu prioritate urgentă;
- Extinderea contorilor APB la 16 biți (pereche de regiștri) pentru a evita wrap-around la 255 pachete;
- Adăugarea unui mecanism de back-pressure: blocarea req_i la source dacă toate FIFO-urile sunt pline;
- Implementarea unui modul de recepție UART (RX) pentru a completa funcționalitatea de comunicare bidirecțională;
- Adăugarea unui mechanism de CRC pe câmpul payload pentru detectarea erorilor de transmisie;
- Formală verificare (Formal Property Verification) cu aserțiunile SVA definite în documentație;
- Creșterea numărului de porturi la 16 prin parametrizarea deplină a modulului (NUM_FIFOS configurabil extern).